

The geometry of Iterative Proportional Scaling for log-linear models in 2D

Pedro N. Jorge

University of Porto

March 21, 2026

Outline

Part 1: Probability Simplex and Statistical Models

Part 2: Log-Linear Model Matrix Representation

Part 3: Classic Iterative Proportional Scaling

Part 4: Next Steps

Outline

Part 1: Probability Simplex and Statistical Models

Part 2: Log-Linear Model Matrix Representation

Part 3: Classic Iterative Proportional Scaling

Part 4: Next Steps

Probability Simplex and Statistical Models

- ▶ Let $\Omega = [n] = \{1, \dots, n\}$ be a discrete sample space. The *probability simplex* Δ_{n-1} is the set of all possible probability distributions on Ω :

$$\Delta_{n-1} = \left\{ (p_1, \dots, p_n) \mid p_i \geq 0, \sum_{i=1}^n p_i = 1 \right\}.$$

- ▶ A *parametric statistical model* $\mathcal{M}_\Theta$ is a family of probability distributions parametrized by $\theta \in \Theta \subseteq \mathbb{R}^d$:

$$\mathcal{M}_\Theta = \{p_\theta \in \Delta_{n-1} \mid \theta \in \Theta\}.$$

- ▶ Therefore we have $\mathcal{M}_\Theta \subseteq \Delta_{n-1}$.

Outline

Part 1: Probability Simplex and Statistical Models

Part 2: Log-Linear Model Matrix Representation

Part 3: Classic Iterative Proportional Scaling

Part 4: Next Steps

Matrix Representation

- ▶ Let $A \in \mathbb{R}^{n \times m}$ with $\mathbf{1}_m \in \text{rowspace}(A)$. Then the matrix A defines a homogeneous polynomial map $\phi_A: \mathbb{R}^n \rightarrow \mathbb{R}^m$ where

$$\phi_A(t_1, \dots, t_n) = \left(\prod_{i=1}^n t_i^{a_{i1}}, \dots, \prod_{i=1}^n t_i^{a_{im}} \right).$$

- ▶ For ϕ_A to be a homogeneous map, $\forall j, \sum \text{col}_j(A)$ is constant, since each coordinate of the map ϕ_A is a monomial.
- ▶ A log-linear model associated to $A \in \mathbb{Z}^{n \times m}$, $\mathcal{M}_A$, is given by

$$\mathcal{M}_A = \text{Im}(\phi_A) \cap \Delta_{m-1}.$$

Proof that $\mathcal{M}_A$ is a Log-Linear Model

► *Proof.* $\mathcal{M}_A = \text{Im}(\phi_A) \cap \Delta_{m-1}$ iff

$$p_j = \prod_{i=1}^n t_i^{a_{ij}} \iff \log p_j = \sum_{i=1}^n c_i a_{ij}, \quad c_i = \log t_i$$

$\therefore \log p = A^T \mathbf{c}$. Since $A^T \mathbf{c} \in \text{colspace}(A^T) = \text{rowspan}(A)$, we conclude that

$$\mathcal{M}_A = \{p \in \Delta_{m-1} \mid \log p \in \text{rowspan}(A)\} \quad \blacksquare$$

Outline

Part 1: Probability Simplex and Statistical Models

Part 2: Log-Linear Model Matrix Representation

Part 3: Classic Iterative Proportional Scaling

Part 4: Next Steps

Multipartition Matrices

- ▶ A matrix $A \in \{0, 1\}^{n \times m}$ is a multipartition matrix if it consists of blocks of rows $A^1, A^2, \dots, A^k$, such that, the sum of each column of A^ℓ is equal to 1.

$$A = A^1 A^2 \dots A^k := \begin{bmatrix} A^1 \\ \hline A^2 \\ \hline \vdots \\ \hline A^k \end{bmatrix}$$

- ▶ Let a_i^ℓ denote the i -th row of the ℓ -th partition. Let n_ℓ denote the number of rows of A^ℓ and

$$\mathcal{I}_i^\ell = \text{supp}(a_i^\ell) = \{j \in [m] \mid (a_i^\ell)_j \neq 0\}$$

Multipartition Matrices

- ▶ Since $A \in \{0, 1\}^{n \times m}$ and the sum of each column of $A^\ell = 1$, for a fixed A^ℓ and index $j \in [m]$, there is *exactly one* row a_i^ℓ with j in its support.
- ▶ We define $S(\ell, j) \in [n_\ell]$ where $S(\ell, j)$ is the index s.t. $j \in \mathcal{I}_{S(\ell, j)}^\ell$.
- ▶ Thus, $a_{S(\ell, j)}^\ell$ is the row of A^ℓ with j in its support.

Maximum Likelihood Estimation (MLE)

- ▶ Let u be the vector of counts of the observed data. Assuming the observations are i.i.d, the likelihood function for u is $L(p \mid u): \mathcal{M}_A \rightarrow \mathbb{R}$

$$L(p \mid u) = \prod_{j=1}^m p_j^{u_j}$$

- ▶ Evaluating the likelihood function at $p \in \mathcal{M}_A$ yields the probability of observing the data u from the distribution p , and so we want to find

$$\text{MLE} := p^* = \arg \max_{p \in \mathcal{M}_A} L(p \mid u)$$

From MLE Characterization to IPS

- ▶ The MLE is the unique solution of the equation

$$Ap = Au.$$

- ▶ For a multipartition matrix, this decomposes into

$$A^i p = A^i u, \quad i = 0(1)k.$$

- ▶ IPS solves $Ap = Au$ cyclically by solving $A^i p = A^i u$ one at a time. Each step is an information projection to the linear family L^i defined by the i -th partition A^i defined as

$$L^i = \{p \in \Delta_{m-1} \mid A^i p = A^i u\}.$$

Classic Iterative Proportional Scaling (IPS)

- ▶ The information projection of p to a non-empty, closed, convex set $M \subset \Delta_{m-1}$ is defined as the element $p' \in M$

$$p' = \arg \min_{q \in M} D_{KL}(q \parallel p) = \arg \min_{q \in M} \sum_{j=1}^m q_j \log \frac{q_j}{p_j}.$$

- ▶ If we project p to L^i we get

$$p'_j = p_j \cdot \frac{a_{S(i,j)}^i \cdot u}{a_{S(i,j)}^i \cdot p}.$$

- ▶ So, the ℓ -th step of the IPS is given as

$$p^\ell = \text{proj}_{L^{\ell'}}(p^{\ell-1}),$$

where $\ell' = \ell \bmod k$, and $p^0 = (\frac{1}{m}, \dots, \frac{1}{m})^T$.

Classic Iterative Proportional Scaling (IPS)

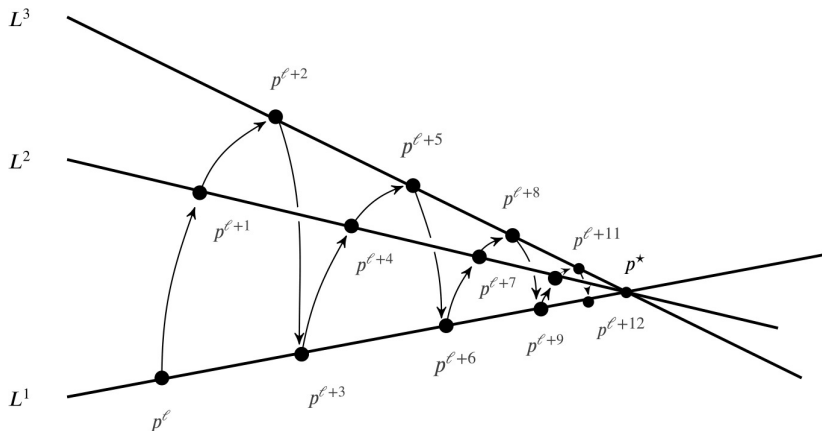


Figure 1: Sketch of the IPS algorithm in the case of three linear families.¹

¹Source: J. I. Coons, C. Langer, and M. Ruddy, "Classical iterative proportional scaling of log-linear models with rational maximum likelihood estimator," *International Journal of Approximate Reasoning*, vol. 164, p. 109043, 2024.

IPS Python3 implementation

```
30 def IPS(A: list[np.ndarray], u: np.ndarray) -> np.ndarray:
29     """
28     Args:
27         A: Multipartition matrix. List of k ndarray matrices of shape [n, m].
26             Note: A[i] can have a different shape in comparison to A[j].
25         u: Vector of counts of the observed data.
24
23     Returns:
22         The MLE,  $p^*$  distribution, of the statistical model associated to A.
21     """
20     k = len(A) # Number of partitions
19     m = u.shape[0] # Distribution  $p^*$  lives in the m-1 simplex
18     l = 0 # Current iteration
17     p_prev = np.full(m, 1/m, dtype=float) #  $p^0$ 
16     p = np.zeros(m)
15
14     MAX_ITER = 100
13     CONVERGENCE_TOL = 1e-12
12     while l < MAX_ITER:
11         p = info_proj_L(p_prev, A[(l-1) % k], u)
10
9         if np.max(np.abs(p - p_prev)) < CONVERGENCE_TOL:
8             print(f"{GREEN}Converged in {l} iterations!{RESET}")
7             return p
6
5             l += 1
4             print(f"{YELLOW}Iteration {l} produced {p} using  $A^{(l-1) \% k + 1}$ {RESET}")
3             p_prev = p.copy()
2         else:
1             print(f"{RED}Could not converge in {l} iterations!{RESET}")
69     return p
```

Figure 2: Code snippet of the IPS implementation in Python3.²

²Code available at:

Outline

Part 1: Probability Simplex and Statistical Models

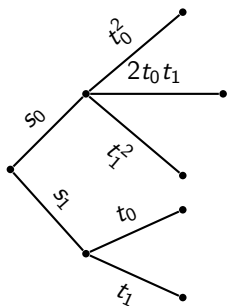
Part 2: Log-Linear Model Matrix Representation

Part 3: Classic Iterative Proportional Scaling

Part 4: Next Steps

Generalized Iterative Scaling (GIS)

- **Objective 1:** Study the *Generalized Iterative Scaling (GIS)* algorithm, as the current classic IPS algorithm only works for 0/1 matrices of constant column sum.
- **Objective 2:** Apply GIS to this binary staged tree



$$A = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 2 & 1 & 0 & 1 & 0 \\ 0 & 1 & 2 & 0 & 1 \end{bmatrix}$$

Figure 3: Staged tree and its associated multipartition matrix

Generalized Iterative Scaling (GIS)

- ▶ **Objective 3:** Implement a symbolic IPS algorithm in Julia that outputs the exact rational formula of the MLE, rather than numerical approximations.
- ▶ **Objective 4:** Better understand the relationship between Newton polytopes and models with rational MLE. Example:

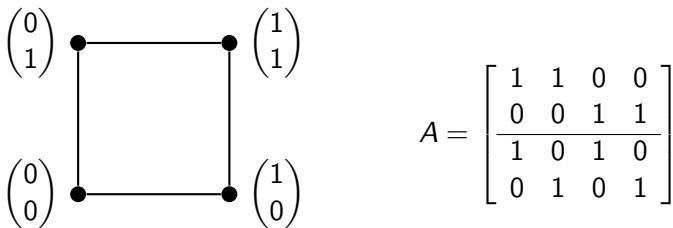


Figure 4: Representation of the binary independence model for two binary random variables via its Newton polytope and its multipartition matrix.